



ODINANN: Direct Insert for Consistently Stable Performance in Billion-Scale Graph-Based Vector Search

Hao Guo Youyou Lu*
Tsinghua University

Abstract

Approximate Nearest Neighbor Search (ANNS) is widely used in various scenarios. For billion-scale ANNS, on-disk graph-based indexes, which organize the vectors as a graph and store them on disk, are favored for their performance and cost-efficiency. However, existing indexes can not maintain a stable search performance while inserting new vectors.

In this paper, we propose to use *direct insert*, which directly inserts vectors into the on-disk index, rather than buffering them in memory and merging them to disk in batches like existing systems. This approach can even out the interference of insert with frontend search, thus stabilizing the performance. We evaluate direct insert by integrating it into a billion-scale graph-based ANNS index named ODINANN. With a fixed insert rate, ODINANN outperforms state-of-the-art ANNS indexes in search latency and throughput, and it consistently shows stable performance in billion-scale vector datasets.

1 Introduction

Approximate nearest neighbor search (ANNS) is the key to multi-modal data retrieval in web search [10, 17] and retrieval-augmented generation (RAG) [15]. Multi-modal data, such as texts and images, are encoded to high-dimensional vectors using neural networks [9, 19]. An ANNS index is built upon these vectors and traversed during a search to get the k nearest neighbors of the query vector. Among all types of ANNS indexes, *on-disk graph-based indexes* [24, 27], in which vectors are organized as a graph and stored on disk, are favored for their performance and cost-efficiency to support large-scale vector search (e.g., billions of vectors [7, 22, 29]).

There is a strong need for ANNS indexes to support vector updates because current systems generate new data continuously [16, 26, 29, 30]. Vector updates can be handled in two ways, index rebuild and index update. Index rebuild, which takes several days in billion-scale datasets [30], fails to keep search results up-to-date. Therefore, index update is favored

by recent on-disk graph-based indexes [23]. They support vector inserts and deletes using *buffered insert and delete*, where they absorb updates into an in-memory index, and then bulk merge the in-memory index into the on-disk index periodically. This way reduces the overhead of index updates by combining disk writes during merge, thus enabling real-time vector search and online index update.

However, by experimental analysis, we find that buffered insert is inefficient when merging inserts to disk. First, merge interferes with frontend search tasks (e.g., increases their median latency to 1.54 $\times$), owing to disk bandwidth interference. Second, merge has a high memory consumption (e.g., 125GB for merging 3% of vectors into a billion-scale index [23]), which consists of the in-memory index for inserts and buffered disk writes during the merge. Third, merge gains little performance boost even with a large insert batch, whose throughput is capped at 3000 QPS in our experiment. This is because merge first finds the neighbors for the inserted vectors using on-disk ANNS. This process can not be batched efficiently and thus is executed one by one.

In this paper, we propose to use *direct insert* for on-disk graph-based ANNS indexes, where vectors are directly inserted into the on-disk index one by one, rather than buffered in memory and merged to disk later as in buffered insert. This approach can even out the insert cost and avoid the in-memory index used by buffered insert, thus stabilizing the frontend search performance and saving memory. Also, it is possible to make direct insert as fast as buffered insert, because buffered insert does not benefit much from batching. However, it is non-trivial to achieve efficient direct insert in graph-based indexes. There are several challenges as follows.

(1) High disk write overhead of direct insert. Direct insert updates tens or hundreds of neighbor records of the target vector. Updating them in place incurs massive disk writes. To combine these record updates for fewer disk writes, a straightforward way is to use a log-structured data layout, but it suffers from garbage collection (GC) overhead.

(2) Complex concurrency control with search. Direct insert searches the target vector to find its neighbors, which

*Youyou Lu is the corresponding author (luyouyou@tsinghua.edu.cn).

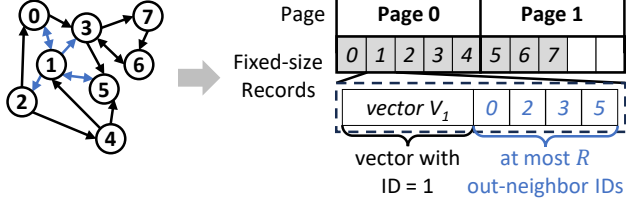


Figure 1: On-disk layout of a graph-based ANNS index. The directed graph is stored as fixed-size records on disk. Each record contains a vector and its out-neighbor IDs (edges). The maximum out-neighbor number of each node is limited to R .

are scattered along the search path. For concurrency control, simply locking all of them before updating causes near-root lock contention. An alternative approach is to leverage the approximate features of insert and search for better concurrency, but how to do this efficiently on disk is still under-exploited.

To address the challenges, we propose ODINANN, a billion-scale graph-based ANNS index with direct inserts. ODINANN achieves efficient direct insert with two techniques as follows.

First, to reduce disk write overhead, we propose *GC-free update combining*. The key observation is that the graph index is stored as fixed-size records on disk. This enables out-of-place record updates to be performed without garbage collection. Therefore, we do space overprovision on disk to reserve multiple free record slots in a page, where multiple record updates can be combined. The old records can be directly recycled without GC. By default, ODINANN doubles disk space to combine updates, resulting in $2\times$ disk writes compared to a log-structured layout.

Second, to increase the concurrency of insert and search, we introduce *approximate concurrency control*, leveraging the approximate features of operations for better parallelism. The key idea is to ensure per-record isolations rather than per-operation ones. For vector search, only a consistent snapshot for each record is required to ensure its correctness. For vector insert, it links the target vector with an approximate neighbor snapshot. Besides, we propose two optimizations for insert, tailored for disk I/O and computing in the critical sections separately, to further improve the concurrency.

We compare ODINANN with state-of-the-art ANNS indexes supporting updates, including graph-based DiskANN [23] and cluster-based SPFresh [30]. In workloads with concurrent inserts and searches, ODINANN shows stable performance, with a median search latency fluctuation¹ of only $1.07\times$, compared to DiskANN's $2.44\times$. Compared to SPFresh, ODINANN achieves 62.1% median search latency and $\sim 15\%$ higher accuracy simultaneously, because of the intrinsic advantage of graph-based indexes over cluster-based ones. In billion-scale datasets, ODINANN simultaneously reaches 5000 QPS search throughput and 1100 QPS insert throughput, with a consis-

¹ Defined as (maximum value / minimum value) in this paper.

Algorithm 1 Vector Search

```

1:  $G \leftarrow$  graph,  $q \leftarrow$  query vector
2: procedure SEARCH( $G, q$ )
3:    $s \leftarrow$  starting vector,  $l \leftarrow$  candidate pool length
4:   candidate pool  $P \leftarrow \{ \langle s.ID, PQ\_distance(s, q) \rangle \}$ 
5:   explored pool  $E \leftarrow \emptyset, i \leftarrow 0$ 
6:   while  $i < l$  do
7:      $i \leftarrow$  index of the first vector in  $P$  but not in  $E$ 
8:      $r \leftarrow$  record with  $ID = P[i].ID$   $\triangleright$  read from disk.
9:      $E.insert(r)$ 
10:    for  $nid$  in  $r.neighbors$  do
11:       $P.insert(\langle nid, PQ\_distance(nid, q) \rangle)$ 
12:    end for
13:     $P \leftarrow l$  nearest vectors to  $q$   $\triangleright$  PQ distance.
14:  end while
15:   $L \leftarrow k$  nearest vectors to  $q$  in  $E$   $\triangleright$  exact distance.
16:  return  $E, L$ 
17: end procedure

```

tently stable median search latency of $\sim 3ms$.

In summary, this paper makes the following contributions:

- We analyze the inefficiency of buffered inserts for on-disk graph-based ANNS indexes (§2).
- We propose ODINANN, a billion-scale graph-based ANNS index with direct inserts. It achieves efficient direct inserts by two key techniques, GC-free update combining, and approximate concurrency control (§3).
- We evaluate ODINANN to show its efficacy in stabilizing the search performance and reducing memory usage during concurrent inserts and searches (§4).

2 Background and Motivation

In this section, we first introduce the layout and operations of on-disk graph-based ANNS indexes. Then, we show the inefficiency of buffered insert and how direct insert overcomes it. Finally, we describe the challenges of direct insert.

2.1 On-Disk Graph-Based ANNS Index

Graph layout. In a graph-based index, vectors are organized as a directed graph, which is stored as adjacent lists on disk, as shown in Figure 1. Each disk page contains several records. Each record consists of a vector (node) and its out-neighbor IDs (edges). To ensure a fixed size for records, the nodes' maximum out-degrees are globally limited to R , where R is a configurable parameter.

Operations. To our knowledge, DiskANN [23] is the only on-disk graph-based ANNS index supporting updates. Here, we introduce its search and buffered insert operations.

Vector search. As shown in Algorithm 1, the search process follows a best-first approach. It starts at a fixed starting vector